

Binary GCD on Unsigned 64-Bit Integers

LeanExe library PoC

September 2026

1 Task and interface

The task is the greatest common divisor of two nonnegative integers represented by `UInt64`. The intended result of `gcdBinary a b` is $\text{gcd}(a, b)$, with $\text{gcd}(0, 0) = 0$. Euclidean GCD and binary GCD are two algorithms for this task. This component implements the latter with shifts and subtraction. Formal correctness proofs for the library integration are deferred.

2 Algorithm and executable source

Common factors of two can be removed from both operands and restored at the end. A factor of two in just one operand can be removed when the other is odd. Subtracting the smaller odd operand from the larger produces an even difference. These identities are described in NIST's binary GCD entry [1].

```
namespace LeanExe.Lib.NumberTheory

def gcdBinary (a b : UInt64) : UInt64 := Id.run do
  if a == 0 then return b
  if b == 0 then return a
  let mut x := a
  let mut y := b
  let mut shift : UInt64 := 0
  while (x ||| y) &&& 1 == 0 do
    x := x >>> 1
    y := y >>> 1
    shift := shift + 1
  while x &&& 1 == 0 do
    x := x >>> 1
  while y != 0 do
    while y &&& 1 == 0 do
      y := y >>> 1
    if x > y then
      let saved := x
      x := y
      y := saved
    y := y - x
  return x <<< shift

end LeanExe.Lib.NumberTheory
```

The document build reads this listing from the executable Lean file. Zero inputs return before a shift loop can encounter zero. The common shift count is at most 63 for nonzero inputs. Each subtraction orders the operands first, so the intended arithmetic uses a nonnegative difference. The returned value fits within the original inputs. The implementation carries three words and allocates no arrays.

3 A shared client task

The example program provides two fraction-reduction functions. One calls Euclidean GCD and the other calls binary GCD. Both use the same result helper, which rejects a zero denominator and divides the numerator and denominator by the selected divisor.

```
tools/seminum gcd-binary 48 18
6
tools/seminum fraction-binary 48 18
8/3
tools/seminum fraction 48 18
8/3
```

Selection occurs through the imported declaration and client call. The catalog associates both implementations with the greatest-common-divisor task and records their distinct algorithms. The compiler’s source-name annotations identify the functions present in the generated WASM.

4 Tests and comparison limits

The test suite applies both GCD algorithms to the same word pairs and compares Wasmtime output with native Lean. Native results also agree with `Nat.gcd` on those cases. Inputs include zero, maximum words, powers of two, large consecutive Fibonacci numbers, and deterministic full-width pairs. Fraction clients cover zero numerators and denominator rejection.

The comparison measures functional agreement on these inputs. Performance comparison requires a separate experiment with repeated calls in one instance so compilation and process startup do not dominate the measurements.

References

- [1] Paul E. Black, “binary GCD,” NIST Dictionary of Algorithms and Data Structures, 2020. <https://xlinux.nist.gov/dads/HTML/binaryGCD.html>. This entry supplies the parity identities and identifies Knuth’s Algorithm 4.5.2B. The implementation here counts shifts and handles zero inputs before the loop.